

Use Case 1: E-Commerce Sales ETL Pipeline

Batch Processing with AWS Glue (PySpark) and Snowflake

1. Overview

In this project, you will build a complete batch ETL (Extract, Transform, Load) pipeline that ingests raw e-commerce sales data from CSV files, transforms and enriches it using PySpark on AWS Glue, and loads the results into a Snowflake data warehouse. This is a foundational pattern used extensively in real-world data engineering at retail and e-commerce companies.

2. Business Scenario

You are a **data engineer at ShopEasy**, a mid-size online retailer. The company collects daily sales data in CSV format from its order management system. Currently, the analytics team has no centralized warehouse and relies on ad-hoc spreadsheet analysis. Your manager has tasked you with building an automated batch ETL pipeline that:

- Ingests raw CSV files (orders, products, customers) from Amazon S3
- Validates and cleans the data
- Joins and enriches orders with product and customer details
- Computes business-critical aggregations (daily sales, top products, customer lifetime value)
- Maintains a slowly changing customer dimension (SCD Type-1)
- Loads all transformed data into Snowflake so the analytics team can run reports and dashboards

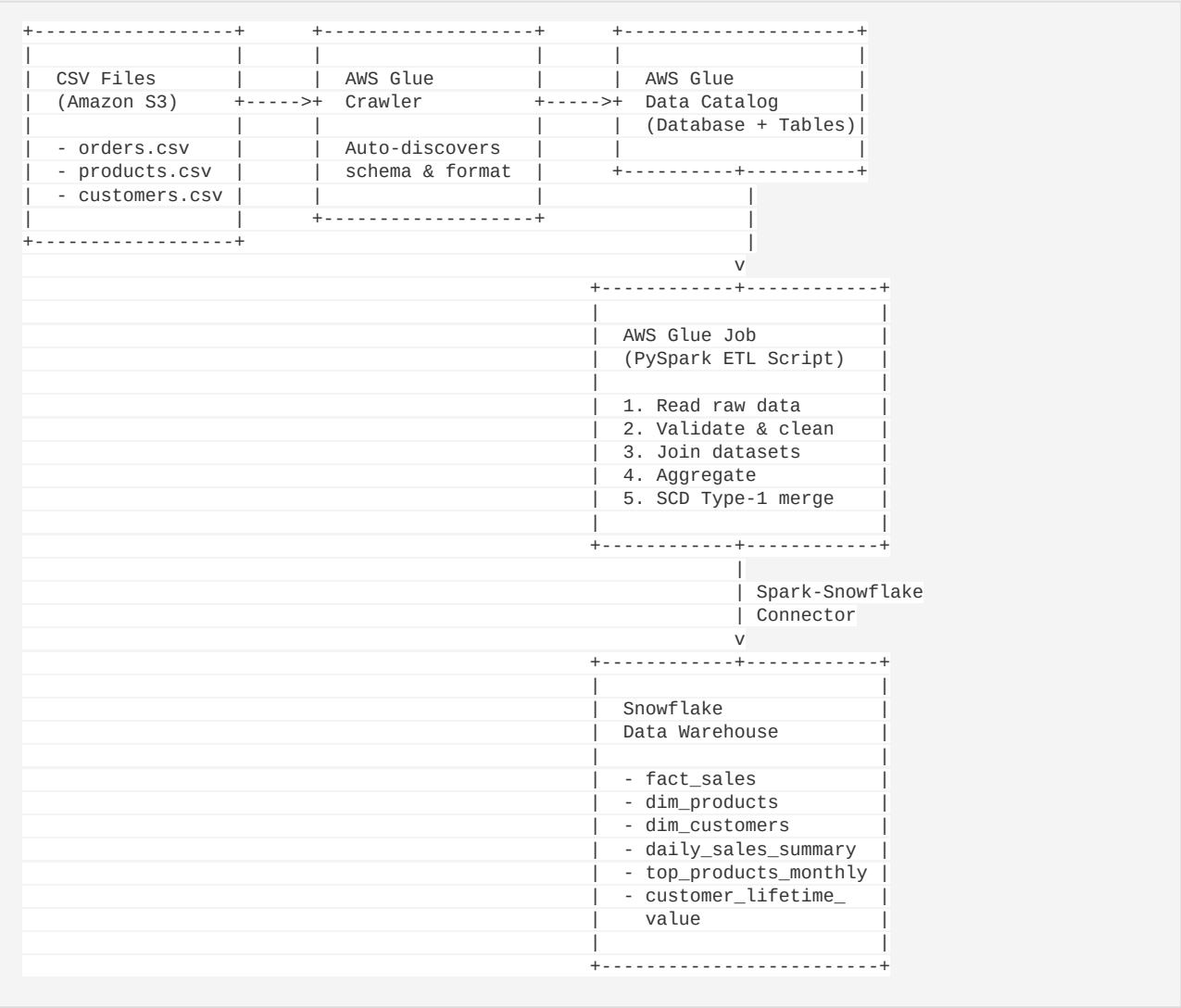
The pipeline should run daily and be idempotent -- running it multiple times with the same input should not create duplicate records.

3. Learning Objectives

By completing this use case, you will learn how to:

1. **Configure AWS Glue Crawlers** to automatically discover and catalog CSV file schemas in the Glue Data Catalog
 2. **Write PySpark transformations** on AWS Glue, including data validation, type casting, deduplication, and joins across multiple datasets
 3. **Build business aggregations** using PySpark (daily sales summaries, top products by revenue, customer lifetime value)
 4. **Implement SCD Type-1 logic** to maintain a slowly changing customer dimension table where updates overwrite old values
 5. **Use the Spark-Snowflake connector** to write DataFrames directly into Snowflake tables from a Glue job
 6. **Schedule and monitor** AWS Glue jobs, including parameterization and error handling best practices
-

4. Architecture Diagram



Data Flow Summary: 1. Raw CSV files are uploaded to an S3 bucket. 2. An AWS Glue Crawler scans the files and populates the Glue Data Catalog with table definitions. 3. An AWS Glue Job (PySpark script) reads the cataloged data, performs transformations, and writes results to Snowflake. 4. The analytics team queries Snowflake for reports and dashboards.

5. Dataset Description

You are provided with three CSV files located in `datasets/use_case_1/`.

5.1 orders.csv (~100 rows)

Column	Data Type	Description
order_id	STRING	Unique order identifier (ORD001 - ORD100)
customer_id	STRING	Foreign key to customers table (CUST001 - CUST050)

product_id	STRING	Foreign key to products table (PROD001 - PROD020)
quantity	INTEGER	Number of units ordered (1-5)
unit_price	DECIMAL	Price per unit at time of order (9.99 - 499.99)
order_date	DATE	Date the order was placed (2024-01-01 to 2024-12-31)
status	STRING	Order status: Completed, Pending, Cancelled, Returned

5.2 products.csv (20 rows)

Column	Data Type	Description
product_id	STRING	Unique product identifier (PROD001 - PROD020)
product_name	STRING	Descriptive product name
category	STRING	Product category (Electronics, Clothing, Home & Kitchen, Sports, Books)
price	DECIMAL	Current listed price
supplier	STRING	Supplier company name

5.3 customers.csv (50 rows)

Column	Data Type	Description
customer_id	STRING	Unique customer identifier (CUST001 - CUST050)
first_name	STRING	Customer first name
last_name	STRING	Customer last name
email	STRING	Customer email address
city	STRING	US city of residence
state	STRING	US state of residence

signup_date	DATE	Date the customer created their account
-------------	------	---

6. Tasks

Work through these tasks in order. Each task builds on the previous one.

Task 1: Set Up Snowflake Database and Tables

Create the target Snowflake environment for your pipeline.

- ☐ Create a database called SHOPEASY_DW
- ☐ Create a schema called ECOMMERCE
- ☐ Create raw staging tables: RAW_ORDERS, RAW_PRODUCTS, RAW_CUSTOMERS
- ☐ Create transformed tables: FACT_SALES, DIM_PRODUCTS, DIM_CUSTOMERS
- ☐ Create aggregation tables: DAILY_SALES_SUMMARY, TOP_PRODUCTS_MONTHLY, CUSTOMER_LIFETIME_VALUE

Task 2: Configure AWS Glue Crawler

Set up automatic schema discovery for your CSV files.

- ☐ Upload the three CSV files to an S3 bucket (e.g., s3://your-bucket/ecommerce/raw/)
- ☐ Create an AWS Glue Crawler pointing to the S3 path
- ☐ Run the crawler and verify that three tables appear in the Glue Data Catalog
- ☐ Inspect the inferred schemas and note any type mismatches

Task 3: Read and Validate Raw Data with PySpark

Write the extraction and validation portion of your Glue job.

- ☐ Initialize GlueContext and SparkSession
- ☐ Read each CSV file into a Spark DataFrame (use the Glue Data Catalog or read directly from S3)
- ☐ Validate data: check for null order_id, null customer_id, negative quantities
- ☐ Cast columns to correct types (e.g., quantity to IntegerType, unit_price to DecimalType)
- ☐ Remove duplicate rows based on primary keys
- ☐ Log row counts before and after validation

Task 4: Join Orders with Products and Customers

Enrich the orders data.

- ☐ Join orders with products on product_id to get product_name, category, supplier
- ☐ Join the result with customers on customer_id to get customer name, city, state

- ☐ Compute `total_amount = quantity * unit_price` for each order line
- ☐ This enriched DataFrame becomes the basis for `FACT_SALES`

Task 5: Create Business Aggregations

Build the summary tables the analytics team needs.

- ☐ **Daily Sales Summary:** Group by `order_date`, compute `total_sales` (sum of `total_amount`), `order_count`, `avg_order_value`
- ☐ **Top Products Monthly:** Group by year-month and product, compute `total_revenue` and `total_quantity_sold`, rank by revenue
- ☐ **Customer Lifetime Value:** Group by customer, compute `total_spend`, `order_count`, `avg_order_value`, `first_order_date`, `last_order_date`

Task 6: Implement SCD Type-1 for Customer Dimension

Maintain a slowly changing dimension for customers.

- ☐ Read the existing `DIM_CUSTOMERS` table from Snowflake (if it exists)
- ☐ Compare incoming customer data with existing records
- ☐ For existing customers: overwrite (update) their city, state, email if changed
- ☐ For new customers: insert them as new rows
- ☐ Add metadata columns: `effective_date`, `last_updated`

Task 7: Write Results to Snowflake

Load all transformed data into Snowflake.

- ☐ Configure Snowflake connection options (account, user, password, database, schema, warehouse)
- ☐ Read connection parameters from Glue job parameters (do NOT hardcode credentials)
- ☐ Write `FACT_SALES` using overwrite mode
- ☐ Write `DIM_PRODUCTS` using overwrite mode
- ☐ Write `DIM_CUSTOMERS` using the SCD Type-1 logic from Task 6
- ☐ Write all three aggregation tables using overwrite mode

Task 8: Schedule and Verify

Deploy and automate your pipeline.

- ☐ Upload the PySpark script to S3
- ☐ Create an AWS Glue Job pointing to the script, configure Glue version and worker count
- ☐ Add job parameters for Snowflake connection details and S3 paths
- ☐ Run the job and monitor it in the Glue console
- ☐ Connect to Snowflake and verify all tables are populated

- [] Run the validation queries (Section 9) to confirm correctness

7. Expected Outputs

After a successful pipeline run, the following tables should exist in Snowflake under `SHOPEASY_DW.ECOMMERCE`:

Table	Expected Row Count (approx.)	Description
FACT_SALES	~100	One row per order with enriched details
DIM_PRODUCTS	20	One row per product
DIM_CUSTOMERS	50	One row per customer with SCD metadata
DAILY_SALES_SUMMARY	~75-85	One row per unique order date
TOP_PRODUCTS_MONTHLY	~80-120	One row per product per month
CUSTOMER_LIFETIME_VALUE	~35-40	One row per customer who placed orders

Sample Query Result -- Daily Sales Summary (top 5 days):

ORDER_DATE	TOTAL_SALES	ORDER_COUNT	AVG_ORDER_VALUE
2024-04-10	149.95	1	149.95
2024-01-07	499.99	1	499.99
2024-05-13	749.97	1	749.97

Sample Query Result -- Customer Lifetime Value (top 3):

CUSTOMER_NAME	TOTAL_SPEND	ORDER_COUNT	AVG_ORDER_VALUE
Lisa Wilson	1,xxx.xx	7	xxx.xx
Robert Johnson	x,xxx.xx	8	xxx.xx
Sarah Davis	x,xxx.xx	7	xxx.xx

(Exact values depend on your transformation logic -- compute them yourself!)

8. Hints

Use these hints progressively. Try to solve each task on your own first before looking at the next hint level.

Task 3: Data Validation

Hint Level 1 (Mild): PySpark has built-in functions for filtering nulls and casting types. Check the `pyspark.sql.functions` module.

Hint Level 2 (Medium): Use `df.filter(col("order_id").isNotNull())` for null checks and `df.withColumn("quantity", col("quantity").cast(IntegerType()))` for casting. Use `df.dropDuplicates(["order_id"])` for deduplication.

Hint Level 3 (Strong): Here is the pattern for validation:

```
orders_df = orders_df.filter(
    (col("order_id").isNotNull()) &
    (col("customer_id").isNotNull()) &
    (col("quantity") > 0)
).dropDuplicates(["order_id"])
```

Task 4: Joining DataFrames

Hint Level 1 (Mild): PySpark joins work similarly to SQL joins. You need two separate join operations chained together.

Hint Level 2 (Medium): Use `orders_df.join(products_df, on="product_id", how="inner")` and then join the result with customers. Be careful about column name conflicts.

Hint Level 3 (Strong):

```
enriched_df = orders_df.join(products_df, on="product_id", how="inner") \
    .join(customers_df, on="customer_id", how="inner") \
    .withColumn("total_amount", col("quantity") * col("unit_price"))
```

Task 5: Aggregations

Hint Level 1 (Mild): Use `groupBy()` followed by `agg()` with functions like `sum()`, `count()`, `avg()`.

Hint Level 2 (Medium): For monthly aggregation, extract year-month using `date_format(col("order_date"), "yyyy-MM")`. Use `Window` functions for ranking.

Hint Level 3 (Strong):

```
daily_summary = enriched_df.groupBy("order_date").agg(
    F.sum("total_amount").alias("total_sales"),
    F.count("order_id").alias("order_count"),
    F.avg("total_amount").alias("avg_order_value")
)
```

Task 6: SCD Type-1

Hint Level 1 (Mild): SCD Type-1 means "overwrite." You need to detect whether a customer already exists and update or insert accordingly.

Hint Level 2 (Medium): Read the existing dimension from Snowflake, do a full outer join with the incoming data on `customer_id`, and use `coalesce()` to pick the newer value.

Hint Level 3 (Strong): If this is the first run and the dimension table is empty, simply insert all rows. For subsequent runs, use a left anti join to find new customers and a join with conditional column selection to handle updates.

Task 7: Snowflake Connector

Hint Level 1 (Mild): The Spark-Snowflake connector requires specific options: `sfURL`, `sfUser`, `sfPassword`, `sfDatabase`, `sfSchema`, `sfWarehouse`.

Hint Level 2 (Medium): Use `df.write.format("snowflake").options(**sf_options).option("dbtable", "TABLE_NAME").mode("overwrite").save()`.

Hint Level 3 (Strong): Read Snowflake credentials from Glue job parameters using `getResolvedOptions(args, ['sf_url', 'sf_user', ...])` to avoid hardcoding.

9. Validation Queries

After your pipeline runs, execute these queries in Snowflake to verify correctness.

```
-- 1. Verify table row counts
SELECT 'FACT_SALES' AS table_name, COUNT(*) AS row_count FROM SHOPEASY_DW.ECOMMERCE.FACT_SALES
UNION ALL
SELECT 'DIM_PRODUCTS', COUNT(*) FROM SHOPEASY_DW.ECOMMERCE.DIM_PRODUCTS
UNION ALL
SELECT 'DIM_CUSTOMERS', COUNT(*) FROM SHOPEASY_DW.ECOMMERCE.DIM_CUSTOMERS
UNION ALL
SELECT 'DAILY_SALES_SUMMARY', COUNT(*) FROM SHOPEASY_DW.ECOMMERCE.DAILY_SALES_SUMMARY
UNION ALL
SELECT 'TOP_PRODUCTS_MONTHLY', COUNT(*) FROM SHOPEASY_DW.ECOMMERCE.TOP_PRODUCTS_MONTHLY
UNION ALL
SELECT 'CUSTOMER_LIFETIME_VALUE', COUNT(*) FROM SHOPEASY_DW.ECOMMERCE.CUSTOMER_LIFETIME_VALUE;

-- 2. Verify fact_sales total_amount calculation
SELECT order_id, quantity, unit_price, total_amount,
       (quantity * unit_price) AS expected_total,
       CASE WHEN total_amount = (quantity * unit_price) THEN 'PASS' ELSE 'FAIL' END AS check_result
FROM SHOPEASY_DW.ECOMMERCE.FACT_SALES
LIMIT 10;

-- 3. Check for orphan records (orders without matching products or customers)
SELECT COUNT(*) AS orphan_orders
FROM SHOPEASY_DW.ECOMMERCE.FACT_SALES f
LEFT JOIN SHOPEASY_DW.ECOMMERCE.DIM_PRODUCTS p ON f.product_id = p.product_id
WHERE p.product_id IS NULL;

-- 4. Verify daily sales summary adds up
SELECT
    SUM(total_sales) AS summary_total
FROM SHOPEASY_DW.ECOMMERCE.DAILY_SALES_SUMMARY;

SELECT
    SUM(total_amount) AS fact_total
FROM SHOPEASY_DW.ECOMMERCE.FACT_SALES;
-- Both values should match

-- 5. Top 5 customers by lifetime value
SELECT *
FROM SHOPEASY_DW.ECOMMERCE.CUSTOMER_LIFETIME_VALUE
ORDER BY total_spend DESC
LIMIT 5;

-- 6. Verify product dimension completeness
SELECT COUNT(DISTINCT product_id) AS products_in_facts
FROM SHOPEASY_DW.ECOMMERCE.FACT_SALES;

SELECT COUNT(*) AS products_in_dim
FROM SHOPEASY_DW.ECOMMERCE.DIM_PRODUCTS;

-- 7. Check SCD Type-1 metadata columns exist
SELECT customer_id, first_name, last_name, city, state, effective_date, last_updated
FROM SHOPEASY_DW.ECOMMERCE.DIM_CUSTOMERS
ORDER BY last_updated DESC
LIMIT 5;

-- 8. Monthly revenue trend
SELECT order_month, SUM(total_revenue) AS monthly_revenue
```



```
FROM SHOPEASY_DW.ECOMMERCE.TOP_PRODUCTS_MONTHLY
GROUP BY order_month
ORDER BY order_month;
```

Additional Notes

- **Idempotency:** Your pipeline should be safe to re-run. Using `overwrite` mode in Snowflake writes ensures this for most tables. For the SCD dimension, your merge logic must handle re-runs gracefully.
- **Error Handling:** Wrap your transformations in `try/except` blocks. Log meaningful error messages. The Glue job should fail cleanly if something goes wrong rather than writing partial data.
- **Performance:** With ~100 rows this dataset is tiny, but write your code as if it could scale to millions. Use proper partitioning and avoid `collect()` on large DataFrames.
- **Security:** Never hardcode Snowflake passwords in your script. Use Glue job parameters or AWS Secrets Manager.

Good luck! This pipeline mirrors real production workloads you will encounter as a data engineer.